

Proyecto Fase 2

ARMv8 Simulator

1. Competencias.....	2
1.1. Competencia General.....	2
1.2. Competencias Específicas.....	2
2. Descripción.....	2
2.1. Componentes de la aplicación.....	3
2.1.1. Tabla de Cuádruplos.....	3
2.1.2. Reporte de CST.....	3
2.2. Flujo del proyecto.....	5
2.3. Arquitectura.....	6
2.3.1. Aplicación Web Estática.....	6
3. Nuevas implementaciones del lenguaje.....	7
3.1 Código Assembler ARMv8.....	7
3.2 Comentarios.....	8
3.3 Registros.....	8
3.3.1 Registros de propósito general:.....	8
3.3.2 Registros de pila:.....	9
3.3.3 Registros de uso reducido:.....	9
3.4 Etiquetas.....	10
3.5 Saltos.....	11
3.5 Operadores.....	12
3.5.1 Operadores Aritméticos:.....	12
3.5.2 Operadores Lógicos:.....	14
3.5.3 Operadores Relacionales:.....	15
3.5.4 Operadores de Desplazamiento:.....	16
3.6 Nuevas implementaciones.....	17
3.6.1 Instrucciones aritméticas:.....	17
3.6.2 Instrucciones de manipulación de bits:.....	18
3.6.3 Instrucciones lógicas y de movimiento:.....	18
3.6.4 Instrucciones de manejo de ramas:.....	18
3.6.5 Instrucciones atómicas:.....	19
3.6.6 Instrucciones condicionales:.....	19

3.6.7 Instrucciones de carga y almacenamiento:	19
3.6.8 Modos de direccionamiento (addr):	20
3.6.9 Operaciones Atómicas (ao):	20
3.6.10 Operando 2 (op2):	20
3.6.11 Registros:	21
3.6.12 Registros de propósito especial:	21
3.6.13 Keys:	21
3.6.14 Instrucciones de suma de comprobación:	22
3.6.15 Instrucciones de carga y almacenamiento con atributos:	22
3.6.16 Códigos de condición (cc):	22
3.6.17 Notas para el conjunto de instrucciones:	23
3.6.18 Instrucciones del sistema:	23
4. Reportes Generales:	24
4.1. Reporte de errores:	24
4.2. Reporte de CST:	24
5. Entregables:	25
6. Restricciones:	25
7. Consideraciones:	25
8. Entrega del proyecto:	26

1. Competencias

1.1. Competencia General

Implementar nuevas funcionalidades al simulador para el lenguaje de programación **ensamblador ARMv8 de 64 bits** que abarque la fase de análisis, comprendiendo la estructura del código fuente utilizando herramientas adecuadas para la generación de analizadores léxicos y sintácticos así como la implementación de cuádruplos mediante una estructura de datos.

1.2. Competencias Específicas

- Utilizar herramientas léxicas y sintácticas, en específico Peggy en Javascript, para implementar el reconocimiento y análisis del código fuente de ARMv8, permitiendo al estudiante comprender la estructura del lenguaje y su sintaxis.
- Ampliar las gramáticas implementadas en la fase anterior del proyecto permitiendo reconocer un mayor número de instrucciones en lenguaje ensamblador.
- Familiarizar al estudiante con las herramientas y estructuras de datos disponibles para la creación de un simulador, proporcionando una comprensión profunda de los procesos involucrados en la traducción y ejecución de programas escritos en lenguaje ensamblador arquitectura ARMv8.

2. Descripción

La presente sección proporciona una visión general de la segunda fase del proyecto que se solicita a los estudiantes, centrándose en la continuación del desarrollo del simulador de lenguaje ensamblador arquitectura ARMv8 64 bits.

Esta fase se centra en la continuación del análisis sintáctico, en el cual se deberá modificar la gramática para poder ampliar el catálogo de instrucciones disponibles en el simulador. También se enfoca en la generación de una estructura la cual contendrá información relevante de cada una de las instrucciones identificadas (cuádruplos).

Los principales cambios que se deberán aplicar en esta fase son los siguientes:

- Ampliación de la gramática.
- Generación de cuádruplos.
- Generación del árbol CST.

2.1. Componentes de la aplicación

El objetivo es continuar con el desarrollo del entorno para la creación de aplicaciones en el lenguaje ensamblador ARMv8. Este entorno, también conocido como Simulador, estará diseñado para analizar el código fuente escrito en ARMv8 y proporcionar herramientas esenciales para facilitar el desarrollo de software.

La aplicación debe contar con los mismos componentes de la fase anterior, un apartado para editar código, un área de consola, una sección de reportes, herramientas de carga y descarga de archivos, etc.

Adicionalmente se deben agregar los siguientes componentes:

2.1.1. Tabla de Cuádruplos

La aplicación deberá contar con un área en donde se detallen los cuádruplos encontrados al momento de analizar el código ensamblador. Esta estructura queda a discreción del estudiante, no necesariamente tiene que ser una tabla, el único requerimiento es que cada uno de los campos esté claramente identificado.

La cantidad de columnas en los campos de los cuádruplos queda a discreción del estudiante, teniendo así la posibilidad de agregar más de 2 operandos (lo cual es necesario para algunas instrucciones del lenguaje ensamblador ARMv8).

2.1.2. Reporte de CST

La aplicación deberá contar con un área en donde se detalle el árbol de sintaxis concreta (cst) que se produjo al momento de analizar el archivo de entrada. Este debe de representarse como un grafo, se recomienda utilizar Graphviz para la implementación de dicho árbol. Sin embargo el estudiante es libre de utilizar la herramienta que crea conveniente.

El Estudiante deberá mostrar los nodos que considere necesarios y se realizarán preguntas al momento de la calificación para que explique su funcionamiento.

Interfaz propuesta: El simulador deberá contar como mínimo con los siguientes elementos mostrados en la siguiente interfaz.

Fase 2

← → ↺

https://www.OLC2.com

Code 1

Code 2

Code 3

ANALYZE

CLEAR

LOAD

DOWNLOAD

CONSOLE

QUADRUPLES

valid code

```

.global _start
.section .text

_start:
    mov    x0, #5
    mov    x1, #3

    add    x2, x0, x1

    mov    w8, #93
    svc    0
  
```

(Interfaz propuesta)

Fase 2

← → ↺

https://www.OLC2.com

Code 1

Code 2

Code 3

ANALYZE

CLEAR

LOAD

DOWNLOAD

CONSOLE

QUADRUPLES

	Op	Arg1	Arg2	Res
1	mov	5	—	x0
2	mov	3	—	x1
3	add	x0	x1	x2
4	mov	93	—	w8

```

.global _start
.section .text

_start:
    mov    x0, #5
    mov    x1, #3

    add    x2, x0, x1

    mov    w8, #93
    svc    0
  
```

(Interfaz propuesta)

2.2. Flujo del proyecto

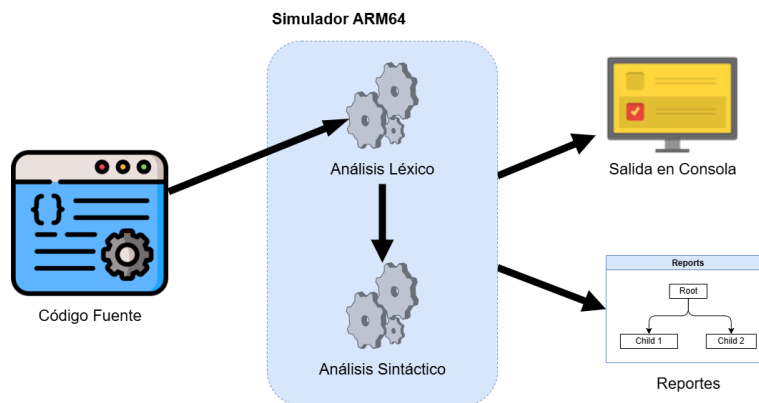
La ejecución del proyecto sigue un flujo definido según los procesos que los usuarios realizan durante su interacción con el sistema. A continuación, se presenta un desglose paso a paso del flujo del proyecto:

1. Inicio del Programa:
 - El usuario inicia el programa desde la aplicación web.
2. Código Fuente:
 - El usuario tiene la opción de cargar un archivo existente que contiene el código ensamblador o crear uno nuevo desde cero.
 - Si elige cargar un archivo existente, se abre el documento en el editor de texto integrado.
 - Si decide crear un nuevo archivo, se inicia un nuevo documento en blanco en el editor.
 - El código fuente puede contener todas las instrucciones disponibles para el lenguaje ensamblador ARMv8 descritas más adelante.
3. Análisis y ejecución: En el caso del simulador contará con las siguientes etapas del compilador:
 - Análisis Léxico: El simulador escanea el código fuente y lo divide en tokens (unidades léxicas como palabras clave, identificadores, operadores, etc.).
 - Análisis Sintáctico: Utiliza la gramática del lenguaje ensamblador ARMv8 para verificar la estructura sintáctica del código y construir un árbol de análisis sintáctico (AST).
4. Validación de Resultados:
 - Se verifica que el programa cuente con la estructura sintáctica correcta, por lo que se debe mostrar un mensaje de validación en la consola de salida, asegurando así la precisión y la funcionalidad del código generado.
5. Generación de Cuádruplos:
 - Se crea la estructura necesaria para mostrar en pantalla los cuádruplos generados a partir de la entrada del código
6. Generación de Reportes:
 - Se generan informes detallados sobre los errores encontrados durante el análisis, incluyendo errores léxicos y sintácticos.
 - Se genera el reporte correspondiente al árbol de sintaxis concreta o CST. Este debe representarse mediante un gráfico con los nodos que el estudiante cree necesarios para cada una de las producciones recorridas en la gramática.

7. Fin del Programa:

- Una vez completadas todas las etapas, el usuario puede cerrar el programa.

Este flujo del proyecto proporciona una guía clara sobre cómo los usuarios interactúan con la aplicación y cómo se llevarán a cabo los procesos clave durante el desarrollo y análisis del código ensamblador ARMv8.



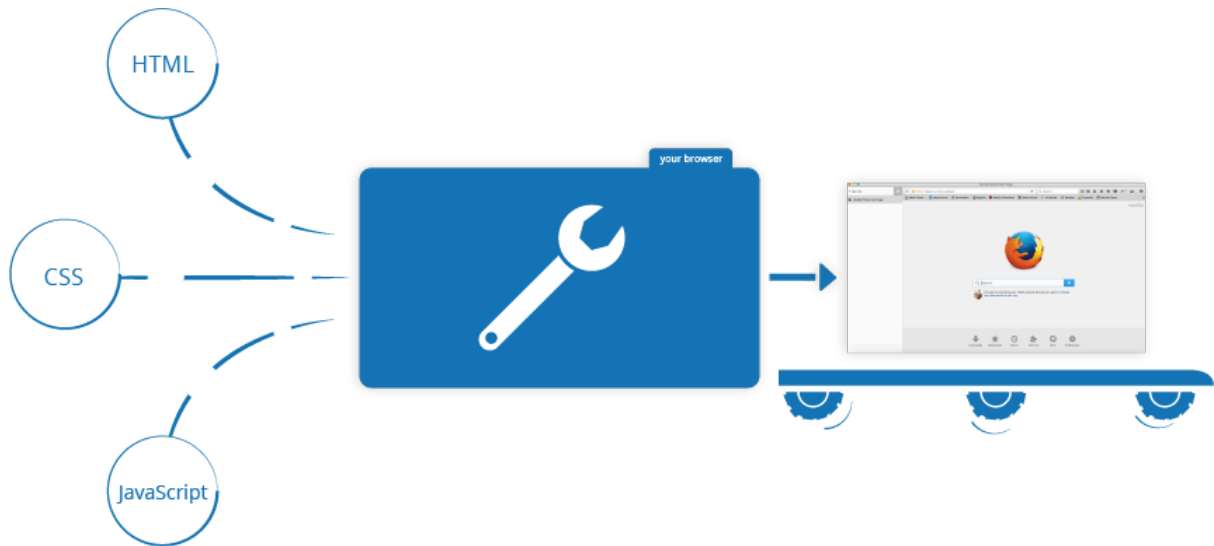
Flujo de ejecución del proyecto

2.3. Arquitectura

Para la implementación de la solución se requiere la siguiente arquitectura:

2.3.1. Aplicación Web Estática

Una aplicación web estática es una aplicación web cuya estructura, contenido y comportamiento no cambian dinámicamente en el lado del servidor. Estas aplicaciones se componen de archivos HTML, CSS y JavaScript que se entregan directamente al navegador del usuario tal como están almacenados en el servidor. Por lo tanto no estará permitido el uso de ningún framework de desarrollo como React, Angular, etc.



3. Nuevas implementaciones del lenguaje

El lenguaje ensamblador que se evaluará a lo largo del proyecto se conforma por un subconjunto de instrucciones de este, con la diferencia de que el utilizado en esta fase tendrá una sintaxis más amplia y sin perder las funcionalidades que caracterizan al lenguaje ensamblador ARMv8 original.

Para esta fase del proyecto se deben agregar nuevas producciones en la gramática, estas producciones deben corresponder a las nuevas instrucciones del lenguaje descritas más adelante. Estas instrucciones corresponden a las descritas por la guía rápida (Quick Reference) de lenguaje ensamblador arquitectura ARMv8 de 64 bits descritos por la universidad de Washington.

3.1 Código Assembler ARMv8

Las nuevas implementaciones del código ensamblador corresponden a las secciones descritas en las primeras dos páginas de la guía rápida de la **Universidad de Washington**, a excepción de las siguientes secciones:

- Cache and TLB Maintenance Instructions.
- DMB and DSB Options.

Para ingresar al documento descrito anteriormente ingresar al siguiente enlace:

- <https://courses.cs.washington.edu/courses/cse469/19wi/arm64.pdf>

Para acceder a la documentación completa de ARMv8 utilizar el siguiente enlace:

- <https://developer.arm.com/documentation/ddi0553/latest/>

Para acceder al listado de llamadas del sistema disponibles utilizar el siguiente enlace:

- https://chromium.googlesource.com/chromiumos/docs/+master/constants/syscalls.md#ARMv8-64_bit

3.2 Comentarios

Un comentario es un componente léxico del lenguaje que no es tomado en cuenta para el análisis de la entrada. Se utilizará el formato del mismo ARMv8 para comentarios:

- Los comentarios de una línea que serán delimitados al inicio con los símbolos “//” y al final con un carácter de finalización de línea.

```
// Este es un comentario en ARMv8

mov X0, #10 // Cargar el valor inmediato 10 en el registro X0

MOV X1, X0 // Cargar el contenido del registro X0 al registro X1
```

En este ejemplo, las líneas que comienzan con // son comentarios y no tienen ningún impacto en el código ensamblador. Los comentarios son útiles para explicar el código y hacer anotaciones.

3.3 Registros

En ensamblador ARMv8, los tipos de datos generalmente se manejan a nivel de registros y direcciones de memoria. No hay tipos de datos explícitos como en lenguajes de más alto nivel. Sin embargo, podemos considerar los registros como almacenando diferentes tipos de datos según su uso.

3.3.1 Registros de propósito general:

ARMv8 cuenta con 31 registros de propósito general, denominados desde x0 hasta x30. Estos registros se utilizan para almacenar datos temporales, direcciones de memoria, valores de retorno de funciones y otros propósitos generales. Los primeros 8 registros (x0 a x7) tienen funciones especiales en algunas instrucciones, como argumentos de función y valores de retorno.

```
# Cargar un entero en el registro x10
mov x10, #42

# Cargar el contenido del registro X10 en el registro X11
```

```
mov x11, x10
```

3.3.2 Registros de pila:

ARMv8 tiene dos registros de puntero de pila. A continuación se muestra un ejemplo simple de cómo se pueden usar estos registros para manipular la pila.

```
.global _start

.section .text

_start:
    // Mover el puntero de la pila para reservar espacio
    sub sp, sp, #16    // Reservar 16 bytes de espacio en la pila

    // Por ejemplo, podemos almacenar un valor en la pila
    mov X0, #42        // Valor a almacenar en la pila
    str X0, [sp]        // Almacenar X0 en la ubicación actual de la pila

    // También podemos cargar un valor de la pila
    ldr X1, [sp]

    // Restaurar el puntero de la pila
    add sp, sp, #16    // Liberar el espacio de la pila

    // Finalmente, podemos salir del programa
    mov X8, #93        // Código de salida del programa (syscall)
    mov X0, #0         // Código de éxito
    mov #0             // Llamada al sistema para salir del programa
```

3.3.3 Registros de uso reducido:

En la arquitectura ARMv8, los registros que contienen la letra "w" están relacionados con operaciones de 32 bits (a diferencia de los registros que contienen la letra "x", que están relacionados con operaciones de 64 bits). Estos registros son conocidos como registros de 32 bits o registros de uso reducido (W-register o Word-register).

```
mov W0, #100 // Primer número (32 bits)

mov W1, #50 // Segundo número (32 bits)
```

3.4 Etiquetas

Las etiquetas serán identificadores únicos que indican una ubicación específica en el código fuente, permitiéndonos realizar saltos a lo largo en el código en la ejecución, estas se definirán durante el proceso de compilación, el formato de la etiqueta está definido de la siguiente manera:

El siguiente ejemplo ilustra un bucle que cuenta desde 10 hasta 1 y luego llama a una subrutina para imprimir el valor actual.

```
.global _start

.section .data
    format_str: .asciz "Current value: %d\n"

.section .text

_start:
    mov x0, #10           // Inicializar el contador a 10

loop:
    cmp x0, #0            // Comparar x0 con 0
    beq end_loop          // Si x0 es 0, saltar al final del bucle

    // Llamar a la subrutina print_value
    bl print_value

    sub x0, x0, #1        // Decrementar el contador
    b loop                // Saltar al inicio del bucle

end_loop:
    // Finalizar el programa y salir
    mov x8, #93           // syscall number para exit
    mov x0, #0            // código de salida
    svc #0                // realizar llamada al sistema para salir
```

```
print_value:
    // Aquí se implementa la lógica para imprimir el valor actual
    ret                // Retorna al llamador
```

Explicación del código:

Inicio del Programa:

- **_start**: es la etiqueta que marca el punto de entrada del programa.
- **mov x0, #10** inicializa el contador en 10.

Bucle:

- **loop**: es la etiqueta que marca el inicio del bucle.
- **cmp x0, #0** compara el valor del contador con 0.
- **beq end_loop** salta a la etiqueta end_loop si el contador es 0.
- **bl print_value** llama a la subrutina print_value para manejar el valor actual del contador.
- **sub x0, x0, #1** decrementa el valor del contador.
- **b loop** salta de nuevo al inicio del bucle para repetir el proceso.

Fin del Bucle:

- **end_loop**: es la etiqueta que marca el final del bucle.
- **mov x8, #93** prepara la llamada al sistema para salir del programa.
- **mov x0, #0** establece el código de salida en 0.
- **svc #0** realiza la llamada al sistema para terminar el programa.

Subrutina print_value:

- **print_value**: es la etiqueta que marca el inicio de la subrutina.
- **ret** retorna al punto donde se llamó a la subrutina.

3.5 Saltos

Los saltos son instrucciones que permiten cambiar el flujo de ejecución del programa. En ARMv8, los saltos condicionales e incondicionales se realizan mediante distintas instrucciones.

Instrucciones de salto condicional: Los saltos condicionales se utilizan para dirigir el flujo del programa en función del resultado de una comparación.

```
.global _start
.section

.text _start:

    mov x0, #10 // Cargar un número en x0
```

```
cmp x0, #0 // Comparar el número con 0

beq zero_label // Si x0 == 0, saltar a zero_label
bgt positive_label // Si x0 > 0, saltar a positive_label
blt negative_label // Si x0 < 0, saltar a negative_label
```

Explicación del código:

Comparación y Saltos Condicionales:

- **cmp x0, #0**: Compara el valor del registro x0 con 0.
- **beq zero_label**: Salta a zero_label si x0 es igual a 0.
- **bgt positive_label**: Salta a positive_label si x0 es mayor que 0.
- **blt negative_label**: Salta a negative_label si x0 es menor que 0.

Instrucciones de salto incondicional: Estas instrucciones siempre realizan un salto a la ubicación especificada, independientemente de alguna condición.

```
.global _start
.section .text

_start:
    mov x0, #5
    add x0, x0, #1    // x0 = x0 + 1

    // Salto incondicional a etiqueta skip_code
    b skip_code
```

3.5 Operadores

A continuación se detallan algunas de las operaciones aritméticas básicas que se pueden realizar en lenguaje ensamblador ARMv8. Dependiendo de la extensión específica y de las características del conjunto de instrucciones, puede haber otras operaciones aritméticas disponibles, pero las principales son las siguientes:

3.5.1 Operadores Aritméticos:

Suma:

```
.global _start

.section .text
```

```
_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #20        // Cargar 20 en x1
    add x2, x0, x1     // x2 = x0 + x1 (suma)
```

Resta:

```
.global _start

.section .text

_start:
    mov x0, #30        // Cargar 30 en x0
    mov x1, #15        // Cargar 15 en x1
    sub x2, x0, x1     // x2 = x0 - x1 (resta)
```

Multiplicación: En este ejemplo, `mul` multiplica el contenido de `x0` y `x1`, y almacena el resultado en `x2`.

```
.global _start

.section .text

_start:
    mov x0, #5         // Cargar 5 en x0
    mov x1, #6         // Cargar 6 en x1
    mul x2, x0, x1     // x2 = x0 * x1 (multiplicación)
```

División: La instrucción `udiv` realiza una división sin signo y `sdiv` realiza una división con signo.

```
. _start

.section .text

_start:
    mov x0, #30        // Cargar 30 en x0
    mov x1, #5         // Cargar 5 en x1
    udiv x2, x0, x1    // x2 = x0 / x1 (división sin signo)
```

3.5.2 Operadores Lógicos:

AND:

```
.global _start

.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mov x1, #0b1010    // Cargar 0b1010 (10 en decimal) en x1
    and x2, x0, x1     // x2 = x0 & x1 (AND)
```

OR:

```
.global _start

.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mov x1, #0b1010    // Cargar 0b1010 (10 en decimal) en x1
    orr x2, x0, x1     // x2 = x0 | x1 (OR)
```

XOR:

```
.global _start

.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mov x1, #0b1010    // Cargar 0b1010 (10 en decimal) en x1
    eor x2, x0, x1     // x2 = x0 ^ x1 (XOR)
```

NOT: La instrucción mvn (Move NOT) invierte todos los bits de un registro.

```
.global _start
```

```
.section .text

_start:
    mov x0, #0b1100    // Cargar 0b1100 (12 en decimal) en x0
    mvn x1, x0         // x1 = ~x0 (NOT)
```

3.5.3 Operadores Relacionales:

Igualdad:

```
.global _start

.section .text

_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #10        // Cargar 10 en x1
    cmp x0, x1         // Comparar x0 con x1
    beq equal_label    // Si x0 == x1, saltar a equal_label

equal_label:
    // Código para cuando x0 es igual a x1
```

Desigualdad:

```
.global _start

.section .text

_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #20        // Cargar 20 en x1
    cmp x0, x1         // Comparar x0 con x1
    bne not_equal_label // Si x0 != x1, saltar a not_equal_label

not_equal_label:
    // Código para cuando x0 es diferente de x1
```

Menor que:


```

.global _start

.section .text

_start:
    mov x0, #10        // Cargar 10 en x0
    mov x1, #20        // Cargar 20 en x1
    cmp x0, x1         // Comparar x0 con x1
    blt less_than_label // Si x0 < x1, saltar a less_than_label

less_than_label:
    // Código para cuando x0 es menor que x1

```

Mayor que:

```

.global _start

.section .text

_start:
    mov x0, #20        // Cargar 20 en x0
    mov x1, #10        // Cargar 10 en x1
    cmp x0, x1         // Comparar x0 con x1
    bgt greater_than_label // Si x0 > x1, saltar a greater_than_label

greater_than_label:
    // Código para cuando x0 es mayor que x1

```

3.5.4 Operadores de Desplazamiento:

Desplazamiento a la izquierda (Logical Shift Left):

```

.global _start

.section .text

_start:
    mov x0, #1        // Cargar 1 en x0
    lsl x1, x0, #3     // x1 = x0 << 3 (desplazar a la izquierda 3 bits)

```

Desplazamiento a la derecha (Logical Shift Right):

```
.global _start

.section .text

_start:
    mov x0, #8          // Cargar 8 en x0
    lsr x1, x0, #2      // x1 = x0 >> 2 (desplazar a la derecha 2 bits)
```

3.6 Nuevas implementaciones

Como se mencionó anteriormente, en esta fase se deberán agregar nuevas instrucciones adicionales a las que fueron agregadas en la fase 1, estas instrucciones están basadas en la guía rápida de ARMv8 de la **Universidad de Washington**. A continuación se detalla un resumen de las nuevas secciones a implementar en esta fase:

3.6.1 Instrucciones aritméticas:

Arithmetic Instructions			
ADC{S}	rd, rn, rm	$rd = rn + rm + C$	
ADD{S}	rd, rn, op2	$rd = rn + op2$	S
ADR	Xd, $\pm rel_{21}$	$Xd = PC + rel^{\pm}$	
ADRP	Xd, $\pm rel_{33}$	$Xd = PC_{63:12}:0_{12} + rel^{\pm}_{33:12}:0_{12}$	
CMN	rd, op2	$rd + op2$	S
CMP	rd, op2	$rd - op2$	S
MADD	rd, rn, rm, ra	$rd = ra + rn \times rm$	
MNEG	rd, rn, rm	$rd = -rn \times rm$	
MSUB	rd, rn, rm, ra	$rd = ra - rn \times rm$	
MUL	rd, rn, rm	$rd = rn \times rm$	
NEG{S}	rd, op2	$rd = -op2$	
NGC{S}	rd, rm	$rd = -rm - \sim C$	
SBC{S}	rd, rn, rm	$rd = rn - rm - \sim C$	
SDIV	rd, rn, rm	$rd = rn \div rm$	
SMADDL	Xd, Wn, Wm, Xa	$Xd = Xa + Wn \times Wm$	
SMNEGL	Xd, Wn, Wm	$Xd = -Wn \times Wm$	
SMSUBL	Xd, Wn, Wm, Xa	$Xd = Xa - Wn \times Wm$	
SMULH	Xd, Xn, Xm	$Xd = (Xn \times Xm)_{127:64}$	
SMULL	Xd, Wn, Wm	$Xd = Wn \times Wm$	
SUB{S}	rd, rn, op2	$rd = rn - op2$	S
UDIV	rd, rn, rm	$rd = rn \div rm$	
UMADDL	Xd, Wn, Wm, Xa	$Xd = Xa + Wn \times Wm$	
UMNEGL	Xd, Wn, Wm	$Xd = -Wn \times Wm$	
UMSUBL	Xd, Wn, Wm, Xa	$Xd = Xa - Wn \times Wm$	
UMULH	Xd, Xn, Xm	$Xd = (Xn \times Xm)_{127:64}$	
UMULL	Xd, Wn, Wm	$Xd = Wn \times Wm$	

3.6.2 Instrucciones de manipulación de bits:

Bit Manipulation Instructions			
BFI	rd, rn, #p, #n	$rd_{p+n-1:p} = rn_{n-1:0}$	
BFXIL	rd, rn, #p, #n	$rd_{n-1:0} = rn_{p+n-1:p}$	
CLS	rd, rn	$rd = \text{CountLeadingOnes}(rn)$	
CLZ	rd, rn	$rd = \text{CountLeadingZeros}(rn)$	
EXTR	rd, rn, rm, #p	$rd = rn_{p-1:0} : rm_{No}$	
RBIT	rd, rn	$rd = \text{ReverseBits}(rn)$	
REV	rd, rn	$rd = \text{BSwap}(rn)$	
REV16	rd, rn	$\text{for}(n=0..1 3) \text{ } rd_{Hn} = \text{BSwap}(rn_{Hn})$	
REV32	Xd, Xn	$Xd = \text{BSwap}(Xn_{63:32}) : \text{BSwap}(Xn_{31:0})$	
{S,U}BFIZ	rd, rn, #p, #n	$rd = rn_{n-1:0}^? \ll p$	
{S,U}BFX	rd, rn, #p, #n	$rd = rn_{p+n-1:p}^?$	
{S,U}XT{B,H}	rd, Wn	$rd = Wn_{No}^?$	
SXTW	Xd, Wn	$Xd = Wn^{\pm}$	

3.6.3 Instrucciones lógicas y de movimiento:

Logical and Move Instructions			
AND{S}	rd, rn, op2	$rd = rn \& op2$	
ASR	rd, rn, rm	$rd = rn \gg rm$	
ASR	rd, rn, #i ₆	$rd = rn \gg i$	
BIC{S}	rd, rn, op2	$rd = rn \& \sim op2$	
EON	rd, rn, op2	$rd = rn \oplus \sim op2$	
EOR	rd, rn, op2	$rd = rn \oplus op2$	
LSL	rd, rn, rm	$rd = rn \ll rm$	
LSL	rd, rn, #i ₆	$rd = rn \ll i$	
LSR	rd, rn, rm	$rd = rn \gg rm$	
LSR	rd, rn, #i ₆	$rd = rn \gg i$	
MOV	rd, rn	$rd = rn$	S
MOV	rd, #i	$rd = i$	
MOVK	rd, #i ₁₆ {, sh}	$rd_{sh+15:sh} = i$	
MOVN	rd, #i ₁₆ {, sh}	$rd = \sim(i^0 \ll sh)$	
MOVZ	rd, #i ₁₆ {, sh}	$rd = i^0 \ll sh$	
MVN	rd, op2	$rd = \sim op2$	
ORN	rd, rn, op2	$rd = rn \sim op2$	
ORR	rd, rn, op2	$rd = rn op2$	
ROR	rd, rn, #i ₆	$rd = rn \ggg i$	
ROR	rd, rn, rm	$rd = rn \ggg rm$	
TST	rn, op2	$rn \& op2$	

3.6.4 Instrucciones de manejo de ramas:

Branch Instructions			
B	rel ₂₈	PC = PC + rel _{27:2} [±] :0 ₂	
Bcc	rel ₂₁	if(cc) PC = PC + rel _{20:2} [±] :0 ₂	
BL	rel ₂₈	X30 = PC + 4; PC += rel _{27:2} [±] :0 ₂	
BLR	Xn	X30 = PC + 4; PC = Xn	
BR	Xn	PC = Xn	
CBNZ	rn, rel ₂₁	if(rn ≠ 0) PC += rel _{21:2} ⁰ :0 ₂	
CBZ	rn, rel ₂₁	if(rn = 0) PC += rel _{21:2} ⁰ :0 ₂	
RET	{Xn}	PC = Xn	
TBNZ	rn, #i, rel ₁₆	if(rn _i ≠ 0) PC += rel _{15:2} [±] :0 ₂	
TBZ	rn, #i, rel ₁₆	if(rn _i = 0) PC += rel _{15:2} [±] :0 ₂	

3.6.5 Instrucciones atómicas:

Atomic Instructions			
CAS{A}{L}	rs, rt, [Xn]	if (rs = [Xn] _N) [Xn] _N = rt	1
CAS{A}{L}{B,H}	Ws, Wt, [Xn]	if (Ws _{N0} = [Xn] _N) [Xn] _N = Wt _{N0}	1
CAS{A}{L}P	rs,rs2,rt,rt2,[Xn]	if (rs2:rs = [Xn] _{2N}) [Xn] _{2N} = rt2:rt	1
LDao{A}{L}{B,H}	Ws, Wt, [Xn]	Wt=[Xn] _N ⁰ ; [Xn] _N =ao([Xn] _N ,Ws _{N0})	1
LDao{A}{L}	rs, rt, [Xn]	rt = [Xn] _N ; [Xn] _N = ao([Xn] _N , rs)	1
STao{A}{L}{B,H}	Ws, [Xn]	[Xn] _N = ao([Xn] _N , Ws _{N0})	1
STao{A}{L}	rs, [Xn]	[Xn] _N = ao([Xn] _N , rs)	1
SWP{A}{L}{B,H}	Ws, Wt, [Xn]	Wt = [Xn] _N ⁰ ; [Xn] _N = Ws _{N0}	1
SWP{A}{L}	rs, rt, [Xn]	rt = [Xn] _N ; [Xn] _N = rs	1

3.6.6 Instrucciones condicionales:

Conditional Instructions			
CCMN	rn, #i ₅ , #f ₄ , cc	if(cc) rn + i; else N:Z:C:V = f	
CCMN	rn, rm, #f ₄ , cc	if(cc) rn + rm; else N:Z:C:V = f	
CCMP	rn, #i ₅ , #f ₄ , cc	if(cc) rn - i; else N:Z:C:V = f	
CCMP	rn, rm, #f ₄ , cc	if(cc) rn - rm; else N:Z:C:V = f	
CINC	rd, rn, cc	if(cc) rd = rn + 1; else rd = rn	
CINV	rd, rn, cc	if(cc) rd = ~rn; else rd = rn	
CNEG	rd, rn, cc	if(cc) rd = -rn; else rd = rn	
CSEL	rd, rn, rm, cc	if(cc) rd = rn; else rd = rm	
CSET	rd, cc	if(cc) rd = 1; else rd = 0	
CSETM	rd, cc	if(cc) rd = ~0; else rd = 0	
CSINC	rd, rn, rm, cc	if(cc) rd = rn; else rd = rm + 1	
CSINV	rd, rn, rm, cc	if(cc) rd = rn; else rd = ~rm	
CSNEG	rd, rn, rm, cc	if(cc) rd = rn; else rd = -rm	

3.6.7 Instrucciones de carga y almacenamiento:

Load and Store Instructions		
LDP	rt, rt2, [addr]	rt2:rt = [addr] _{2N}
LDPSW	Xt, Xt2, [addr]	Xt = [addr] ₃₂ [±] ; Xt2 = [addr+4] ₃₂ [±]
LD{U}R	rt, [addr]	rt = [addr] _N
LD{U}R{B,H}	Wt, [addr]	Wt = [addr] _N ⁰
LD{U}RS{B,H}	rt, [addr]	rt = [addr] _N [±]
LD{U}RSW	Xt, [addr]	Xt = [addr] ₃₂ [±]
PRFM	prfop, addr	Prefetch(addr, prfop)
STP	rt, rt2, [addr]	[addr] _{2N} = rt2:rt
ST{U}R	rt, [addr]	[addr] _N = rt
ST{U}R{B,H}	Wt, [addr]	[addr] _N = Wt _{N0}

3.6.8 Modos de direccionamiento (addr):

Addressing Modes (addr)		
xxP,LDPSW	[Xn{, #i _{7+s} }]	addr = Xn + i _{6+s:s} [±] :0 _s
xxP,LDPSW	[Xn], #i _{7+s}	addr=Xn; Xn+=i _{6+s:s} [±] :0 _s
xxP,LDPSW	[Xn, #i _{7+s}]!	Xn+=i _{6+s:s} [±] :0 _s ; addr=Xn
xxR*,PRFM	[Xn{, #i _{12+s} }]	addr = Xn + i _{11+s:s} ⁰ :0 _s
xxR*	[Xn], #i ₉	addr = Xn; Xn += i [±]
xxR*	[Xn, #i ₉]!	Xn += i [±] ; addr = Xn
xxR*,PRFM	[Xn,Xm{, LSL #0 s}]	addr = Xn + Xm ≪ s
xxR*,PRFM	[Xn,Wm,{S,U}XTW{ #0 s}]	addr = Xn + Wm [?] ≪ s
xxR*,PRFM	[Xn,Xm,SXTX{ #0 s}]	addr = Xn + Xm [±] ≪ s
xxUR*,PRFM	[Xn{, #i ₉ }]	addr = Xn += i [±]
LDR{SW},PRFM	±rel ₂₁	addr = PC + rel _{20,2} [±] :0 ₂

3.6.9 Operaciones Atómicas (ao):

Atomic Operations (ao)	
ADD [Xn] + rs	SMAX [Xn] > rs ? [Xn] : rs
CLR [Xn] & ~rs	SMIN [Xn] < rs ? [Xn] : rs
EOR [Xn] ⊕ rs	UMAX [Xn] > rs ? [Xn] : rs
SET [Xn] rs	UMIN [Xn] < rs ? [Xn] : rs

3.6.10 Operando 2 (op2):

Operand 2 (op2)		
all	<i>rm</i>	<i>rm</i>
all	<i>rm</i> , LSL $\#i_6$	$rm \ll i$
all	<i>rm</i> , LSR $\#i_6$	$rm \gg i$
all	<i>rm</i> , ASR $\#i_6$	$rm \ggg i$
logical	<i>rm</i> , ROR $\#i_6$	$rm \ggg i$
arithmetic	<i>Wm</i> , {S,U}XTB{ $\#i_3$ }	$Wm_{80}^7 \ll i$
arithmetic	<i>Wm</i> , {S,U}XTH{ $\#i_3$ }	$Wm_{H0}^7 \ll i$
arithmetic	<i>Wm</i> , {S,U}XTW{ $\#i_3$ }	$Wm^7 \ll i$
arithmetic	<i>Xm</i> , {S,U}XTX{ $\#i_3$ }	$Xm^7 \ll i$
arithmetic	$\#i_{12}$	i^0
arithmetic	$\#i_{24}$	$i_{23:12}^0:0_{12}$
AND,EOR,ORR,TST	$\#mask$	<i>mask</i>

3.6.11 Registros:

Registers	
X0-X7	Arguments and return values
X8	Indirect result
X9-X15	Temporary
X16-X17	Intra-procedure-call temporary
X18	Platform defined use
X19-X28	Temporary (must be preserved)
X29	Frame pointer (must be preserved)
X30	Return address
SP	Stack pointer
XZR	Zero
PC	Program counter

3.6.12 Registros de propósito especial:

Special Purpose Registers		
SPSR_EL{1..3}	Process state on exception entry to EL{1..3}	64
ELR_EL{1..3}	Exception return address from EL{1..3}	
SP_EL{0..2}	Stack pointer for EL{0..2}	64
SPSel	SP selection (0: SP=SP_EL0, 1: SP=SP_ELn)	
CurrentEL	Current Exception level (at bits 3..2)	RO
DAIF	Current interrupt mask bits (at bits 9..6)	
NZCV	Condition flags (at bits 31..28)	
FPCR	Floating-point operation control	
FPSR	Floating-point status	

3.6.13 Keys:

Keys	
N	Operand bit size (8, 16, 32 or 64)
s	Operand log byte size (0=byte,1=hword,2=word,3=dword)
rd, rn, rm, rt	General register of either size (Wn or Xn)
prfop	P{LD,LI,ST}L{1..3}{KEEP,STRM}
{,sh}	Optional halfword left shift (LSL # {16,32,48})
val [±] , val ⁰ , val [?]	Value is sign/zero extended (? depends on instruction)
$\bar{x}$ $\div$ $\gg$ $\geq$ $\leq$	Operation is signed

3.6.14 Instrucciones de suma de comprobación:

Checksum Instructions		
CRC32{B,H}	Wd, Wn, Wm	Wd=CRC32(Wn,0x04c11db7,Wm _{N0})
CRC32W	Wd, Wn, Wm	Wd = CRC32(Wn,0x04c11db7,Wm)
CRC32X	Wd, Wn, Xm	Wd = CRC32(Wn,0x04c11db7,Xm)
CRC32C{B,H}	Wd, Wn, Wm	Wd=CRC32(Wn,0x1edc6f41,Wm _{N0})
CRC32CW	Wd, Wn, Wm	Wd = CRC32(Wn,0x1edc6f41,Wm)
CRC32CX	Wd, Wn, Xm	Wd = CRC32(Wn,0x1edc6f41,Xm)

3.6.15 Instrucciones de carga y almacenamiento con atributos:

Load and Store Instructions with Attribute		
LD{A}XP	rt, rt2, [Xn]	rt:rt2 = [Xn, <SetExclMonitor>] _{2N}
LD{A}{X}R	rt, [Xn]	rt = [Xn, <SetExclMonitor>] _N
LD{A}{X}R{B,H}	Wt, [Xn]	Wt = [Xn, <SetExclMonitor>] _N ⁰
LDNP	rt,rt2,[Xn{, #i _{7+s} }]	rt2:rt = [Xn + i _{6+s:s} [±] :0 _s , <Temp>] _{2N}
LDTR	rt, [Xn{, #i ₉ }]	rt = [Xn += i [±] , <Unpriv>] _N
LDTR{B,H}	Wt, [Xn{, #i ₉ }]	Wt = [Xn += i [±] , <Unpriv>] _N ⁰
LDTRS{B,H}	rt, [Xn{, #i ₉ }]	rt = [Xn += i [±] , <Unpriv>] _N [±]
LDTRSW	Xt, [Xn{, #i ₉ }]	Xt = [Xn += i [±] , <Unpriv>] ₃₂ [±]
STLR	rt, [Xn]	[Xn, <Release>] _N = rt
STLR{B,H}	Wt, [Xn]	[Xn, <Release>] _N = Wt _{N0}
ST{L}XP	Wd, rt, rt2, [Xn]	[Xn, <Excl>] _{2N} =rt:rt2; Wd=fail?1:0
ST{L}XR	Wd, rt, [Xn]	[Xn, <Excl>] _N =rt; Wd=fail?1:0
ST{L}XR{B,H}	Wd, Wt, [Xn]	[Xn, <Excl>] _N =Wt _{N0} ; Wd=fail?1:0
STNP	rt,rt2,[Xn{, #i _{7+s} }]	[Xn + i _{6+s:s} [±] :0 _s , <Temp>] _{2N} = rt2:rt
STTR	rt, [Xn{, #i ₉ }]	[Xn += i [±] , <Unpriv>] _N = rt
STTR{B,H}	Wt, [Xn{, #i ₉ }]	[Xn += i [±] , <Unpriv>] _N = Wt _{N0}

3.6.16 Códigos de condición (cc):

Condition Codes (cc)		
EQ	Equal	Z
NE	Not equal	!Z
CS/HS	Carry set, Unsigned higher or same	C
CC/LO	Carry clear, Unsigned lower	!C
MI	Minus, Negative	N
PL	Plus, Positive or zero	!N
VS	Overflow	V
VC	No overflow	!V
HI	Unsigned higher	C & !Z
LS	Unsigned lower or same	!C Z
GE	Signed greater than or equal	N = V
LT	Signed less than	N ≠ V
GT	Signed greater than	!Z & N = V
LE	Signed less than or equal	Z N ≠ V
AL	Always (default)	1

3.6.17 Notas para el conjunto de instrucciones:

Notes for Instruction Set
S SP/WSP may be used as operand(s) instead of XZR/WZR
1 Introduced in ARMv8.1

3.6.18 Instrucciones del sistema:

System Instructions		
AT	S1{2}E{0..3}{R,W}, Xn	PAR_EL1 = AddrTrans(Xn)
BRK	#i ₁₆	SoftwareBreakpoint(i)
CLREX	{#i ₄ }	ClearExclusiveLocal()
DMB	barrierop	DataMemoryBarrier(barrierop)
DSB	barrierop	DataSyncBarrier(barrierop)
ERET		PC=ELR_ELn;PSTATE=SPSR_ELn
HVC	#i ₁₆	CallHypervisor(i)
ISB	{SY}	InstructionSyncBarrier(SY)
MRS	Xd, sysreg	Xd = sysreg
MSR	sysreg, Xn	sysreg = Xn
MSR	SPSel, #i ₁	PSTATE.SP = i
MSR	DAIFSet, #i ₄	PSTATE.DAIF = i
MSR	DAIFClr, #i ₄	PSTATE.DAIF &= ~i
NOP		
SEV		SendEvent()
SEVL		EventRegisterSet()
SMC	#i ₁₆	CallSecureMonitor(i)
SVC	#i ₁₆	CallSupervisor(i)
WFE		WaitForEvent()
WFI		WaitForInterrupt()
YIELD		

4. Reportes Generales

Como se indicaba al inicio, el simulador genera una serie de reportes sobre el proceso de análisis de los archivos de entrada. Los reportes son los siguientes:

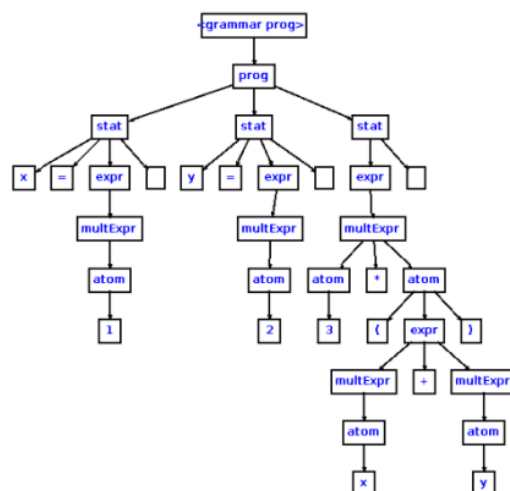
4.1. Reporte de errores

El simulador deberá ser capaz de detectar todos los errores que se encuentren durante el proceso de compilación. Todos los errores se deberán de recolectar y se mostrará un reporte de errores en el que, como mínimo, debe mostrarse el tipo de error, su ubicación y una breve descripción de por qué se produjo.

No.	Descripción	Línea	Columna	Tipo
1	Se esperaba "ID" pero se obtuvo "(".	5	1	sintáctico
2	Se esperaba "ADD", "SUB" o "MUL" pero se obtuvo "}".	19	6	sintáctico
3	El símbolo "-" no es aceptado en el lenguaje.	55	2	léxico

4.2. Reporte de CST

Este reporte mostrará el árbol CST generado al momento de ejecutar el análisis sintáctico del código de entrada. El árbol debe mostrarse de manera gráfica, con cada una de sus hojas y ramas debidamente identificadas.



5. Entregables

El estudiante deberá entregar únicamente el link de un repositorio en GitHub, el cual será privado y contendrá todos los archivos necesarios para la ejecución de la aplicación, así como el código fuente y la gramática. El estudiante es responsable de verificar el contenido de los entregables, los cuales son:

- 5.1. Código fuente de la aplicación
- 5.2. Gramáticas utilizadas

Se deben conceder los permisos necesarios al auxiliar para acceder a dicho repositorio. El usuario es el siguiente:

- AndresRodas

6. Restricciones

- 6.1. El proyecto deberá realizarse como una aplicación web estática utilizando el lenguaje **Javascript**.
- 6.2. Es válido el uso de cualquier herramienta y/o librería en Javascript para el desarrollo de la interfaz gráfica a excepción de frameworks que afecten a la arquitectura del proyecto como Angular, Vue.js, React, etc.
- 6.3. Para el analizador léxico y sintáctico se debe implementar una gramática con la herramienta **Peggy** de Javascript.
- 6.4. Para la validación del código ensamblador se debe tener como base la arquitectura ARMv8.
- 6.5. Las copias de proyectos tendrán de manera automática una nota de **0 puntos** y los involucrados serán reportados a la Escuela de Ciencias y Sistemas.
- 6.6. El desarrollo y la entrega del proyecto es en grupos con un máximo de 3 estudiantes.
- 6.7. La distribución de los grupos queda a discreción de los estudiantes.
- 6.8. El sistema operativo queda a elección del estudiante.

7. Consideraciones

- 7.1. Durante la calificación se realizarán preguntas sobre el código para verificar la autoría de este, de no responder correctamente la mayoría de las preguntas **el tutor hará una verificación exhaustiva en busca de copias**.
- 7.2. Se necesita que los estudiantes al momento de la calificación tengan el entorno de desarrollo y las herramientas necesarias para realizar compilaciones, ya que se pedirán pequeñas modificaciones en el código para verificar la autoría de este, en caso que los estudiantes no puedan realizar dichas modificaciones en un tiempo prudencial, los estudiantes tendrán 0 en

la sección ponderada y ***el tutor hará una verificación exhaustiva en busca de copias.***

- 7.3. Se tendrá un máximo de 30 minutos por estudiante para calificar el proyecto. La calificación será de manera virtual y se grabará para tener constancia o verificación posterior.
- 7.4. La hoja de calificación describe cada aspecto a calificar, por lo tanto, si la funcionalidad a calificar falla en la sección indicada se tendrá 0 puntos en esa funcionalidad y esa nota no podrá cambiar si dicha funcionalidad funciona en otra sección.
- 7.5. Si una función del programa ya ha sido calificada, esta no puede ser penalizada si en otra sección la función falla o es errónea.
- 7.6. Los archivos de entrada permitidos en la calificación son únicamente los archivos preparados por el tutor.
- 7.7. Los archivos de entrada podrán ser modificados solamente antes de iniciar la calificación eliminando funcionalidades que los estudiantes indiquen que no desarrollaron.
- 7.8. La sintaxis descrita en este documento son con fines descriptivos, el estudiante es libre de diseñar la gramática que crea apropiada para el reconocimiento del lenguaje ensamblador ARMv8.

8. Entrega del proyecto

- 8.1. La entrega se realizará de manera virtual, se habilitará un apartado en la plataforma de UEDI para que el estudiante realice su entrega.
- 8.2. No se recibirán proyectos fuera de la fecha y hora estipulada.
- 8.3. La entrega de cada una de las fases del proyecto es en **grupos con máximo de 3 alumnos.**
- 8.4. Fecha límite de entrega del proyecto: **21 de junio de 2024 a las 23:59 hrs.**